

JavaScript - pitanja i mini zadaci

JS01 - Uvod u programiranje i JavaScript

Pitanja

1. Što je programiranje i koji je njegov osnovni cilj?
2. Što je programski jezik?
3. Koja je razlika između imperativnih i deklarativnih programskih jezika?
4. Kojim skupinama pripadaju proceduralno i objektno orijentirano programiranje?
5. Koja je razlika između kompiliranog i interpretiranog izvođenja programa?
6. Tko je stvorio JavaScript i koje godine?
7. Kako se JavaScript zvao prije nego što je dobio današnje ime?
8. Što je ECMAScript i zašto je ES6 važan za moderni JavaScript?
9. Koja je razlika između ispisa pomoću `console.log()`, `alert()` i promjene sadržaja DOM-a?
10. Kada koristimo jednostruke navodnike, dvostruke navodnike i backtick znakove?

Mini zadatak

Napravi HTML stranicu s gumbom i poljem za unos imena. Nakon klika na gumb pročitaj ime i u konzoli ispiši poruku koristeći template literal, primjerice: `Pozdrav, Luka!`. Ako je polje prazno, prikaži upozorenje pomoću `alert()`.

JS02 - Ulaz, izlaz, varijable i tipovi podataka

Pitanja

1. Koju funkciju u pregledniku možemo koristiti za unos podatka od korisnika?
2. Kojeg je tipa vrijednost koju vraća `prompt()`, čak i kada korisnik upiše broj?
3. Zašto je prije računanja često potrebno koristiti `Number()` ili `parseFloat()`?
4. Koja je razlika između funkcija `Number()`, `parseInt()` i `parseFloat()`?
5. Kada treba koristiti `const`, a kada `let`?
6. Zašto se uporaba ključne riječi `var` uglavnom ne preporučuje u modernom JavaScriptu?
7. Što operator `typeof` provjerava?
8. Koji su primitivni tipovi podataka u JavaScriptu?
9. Koja je razlika između vrijednosti `null` i `undefined`?
10. Zašto dva simbola stvorena izrazom `Symbol('id')` nisu jednaka?

Mini zadatak

Pomoću `prompt()` zatraži cijenu proizvoda i postotak popusta. Pretvori oba unosa u brojeve, izračunaj sniženu cijenu i prikaži je korisniku. Varijable koje se ne mijenjaju deklariraj pomoću `const`.

JS03 - JavaScript operatori

Pitanja

1. Čemu služi operator `%` i što vraća izraz `7 % 2`?
2. Koja je razlika između operatora `=` i `===`?
3. Koja je razlika između usporedbi `==` i `===`?
4. Kako rade operatori dodjele `+=` i `--`?
5. Koja je razlika između operatora `&&`, `||` i `!`?
6. Koja je razlika između izraza `i++` i `++i`?
7. Kada je ternarni operator prikladna zamjena za `if/else`?
8. Čemu služi spread operator `...` kod nizova?
9. Kako rade nullish coalescing (`??`) i optional chaining (`?.`)?
10. Čemu služe operatori `typeof`, `instanceof` i `in`?

Mini zadatak

Deklariraj objekt `korisnik` koji može, ali ne mora imati svojstvo `adresa`. Sigurno pročitaj grad pomoću optional chaininga, a ako grad ne postoji, pomoću operatora `??` postavi vrijednost `Nepoznat grad`. Zatim ternarnim operatorom ispiši je li korisnik punoljetan.

JS04 - If uvjetno grananje

Pitanja

1. Kada se izvršava kod unutar osnovnog `if` bloka?
2. Koja je svrha `else` grane?
3. Kako JavaScript odabire granu u strukturi `if - else if - else`?
4. Zašto je redoslijed uvjeta važan kod više `else if` grana?
5. Što je ugniježđeni `if` i kada ga možemo koristiti?
6. Kako se u uvjetima koriste operatori `&&`, `||` i `!`?
7. Koje su najčešće falsy vrijednosti u JavaScriptu?
8. Što je early return i kako može pojednostaviti funkciju?
9. Kada je dobro koristiti ternarni operator umjesto `if/else` strukture?
10. Što block scope znači za varijable deklarirane pomoću `let` i `const`?

Mini zadatak

Napiši funkciju `odrediOcjenu(bodovi)` koja early return obrascem vraća ocjenu: `Odličan` za najmanje 90 bodova, `Vrlo dobar` za najmanje 75, `Dobar` za najmanje 60, a inače `Nedovoljan`. Prije toga provjeri je li broj bodova između 0 i 100.

JS05 - Switch naredba

Pitanja

1. Kako `switch` uspoređuje vrijednost izraza s vrijednostima u `case` granama?
2. Zašto se tipovi vrijednosti u `switch` izrazu i `case` grani moraju podudarati?
3. Čemu služi naredba `break` unutar `case` grane?
4. Kada se izvršava `default` grana?
5. Može li `switch` raditi s vrijednostima tipa `String`, `Boolean`, `null` i `undefined`?
6. Kako se `Symbol` i `BigInt` vrijednosti mogu koristiti u `switch` naredbi?
7. Što `switch` uspoređuje kada su `case` vrijednosti objekti ili nizovi?
8. Zašto dva odvojeno stvorena niza `[1, 2]` nisu ista vrijednost za `switch`?
9. Kako se funkcija može koristiti kao `case` vrijednost?
10. Kako radi obrazac `switch(true)` i za kakve je provjere koristan?

Mini zadatak

Zatraži od korisnika broj mjeseca od 1 do 12. Pomoću `switch` naredbe ispiši naziv godišnjeg doba kojem mjesec pripada. Za neispravan broj ispiši poruku `Nepoznat mjesec`.

JS06 - Osnovni nizovi

Pitanja

1. Kako se deklarira prazan niz, a kako niz s početnim vrijednostima?
2. Od kojeg indeksa počinje numeriranje elemenata niza?
3. Što dobivamo kada pristupimo indeksu koji ne postoji u nizu?
4. Što predstavlja svojstvo `length` i kako se može koristiti za dodavanje elementa na kraj?
5. Što se događa kada ručno smanjimo vrijednost `length` svojstva niza?
6. Što je destructuring niza i kako možemo preskočiti pojedini element?
7. Čemu služe zadane vrijednosti pri destructuringu?
8. Kako spread operatorom kopiramo ili spajamo nizove?
9. Koja je razlika između `const b = a` i `const b = [...a]`?
10. Kako se pristupa elementu višedimenzionalnog niza?

Mini zadatak

Napravi niz od najmanje pet brojeva. Destructuringom izdvoji prvi element, a ostale spremi u novi niz pomoću rest operatora. Zatim napravi novu kopiju niza u kojoj je prvi element zamijenjen nulom, bez mijenjanja originalnog niza.

JS07 - For petlja

Pitanja

1. Kada je klasična `for` petlja dobar izbor?
2. Koja tri dijela obično sadrži zaglavlje `for` petlje?
3. Kako pomoću `for` petlje prolazimo kroz znakove stringa?
4. Kako ispravno prolazimo kroz sve elemente niza pomoću indeksa?
5. Kako se piše petlja koja broji unatrag?
6. Što radi naredba `continue`?
7. Što su ugniježdene petlje i za koje su strukture korisne?
8. Što radi naredba `break`?
9. Čemu služi labela uz petlju i kako utječe na `break` ili `continue`?
10. Kako nastaje beskonačna `for` petlja i kako je možemo zaustaviti?

Mini zadatak

Pomoću dviju ugnijeđenih `for` petlji ispiši tablicu množenja od 1 do 5. Ako je rezultat jednak 12, preskoči njegov ispis pomoću `continue`.

JS08 - While petlja

Pitanja

1. Koja je glavna razlika između `for` i `while` petlje?
2. Kada `while` petlja provjerava svoj uvjet?
3. Koliko će se puta izvršiti tijelo petlje ako je početni uvjet odmah `false`?
4. Zašto unutar `while` petlje moramo mijenjati vrijednost koja utječe na uvjet?
5. Čemu služe `console.time()` i `console.timeEnd()`?
6. Zašto je `while` prikladan za pražnjenje reda čekanja?
7. Što radi metoda `shift()` nad nizom?
8. Zašto je bacanje kocke do pojave šestice dobar primjer uporabe `while` petlje?
9. Kako `continue` može slučajno uzrokovati beskonačnu `while` petlju?
10. Kako se namjerna petlja `while (true)` može sigurno prekinuti?

Mini zadatak

Napravi niz s pet zadataka. Pomoću `while` petlje uklanjaj prvi zadatak metodom `shift()` i ispisuj što se trenutno izvršava te koliko je zadataka ostalo. Petlja završava kada niz postane prazan.

JS09 - Funkcije

Pitanja

1. Što je deklaracija funkcije i što znači da je hoistana?
2. Po čemu se function expression razlikuje od deklaracije funkcije?
3. Koje su glavne značajke arrow funkcije?
4. Što znači da arrow funkcija nema vlastiti `this`?
5. Što je higher-order funkcija?
6. Što je callback funkcija?
7. Što je closure i koje vrijednosti unutarnja funkcija pamti?
8. Kako metoda objekta pristupa svojstvima tog objekta?
9. Koja je uloga konstruktora i instance metode unutar klase?
10. Što uvijek vraća funkcija označena ključnom riječi `async`?

Mini zadatak

Napiši funkciju `napraviMnozitelj(faktor)` koja vraća novu funkciju. Vraćena funkcija prima jedan broj i množi ga sa zapamćenim faktorom. Napravi funkcije `duplo` i `troduplo` te ih isprobaj s nekoliko brojeva.

JS10 - Try, catch, finally i greške

Pitanja

1. Koja je svrha `try` bloka?
2. Kada se izvršava `catch` blok?
3. Kako `catch` sprječava da neobrađena greška prekine ostatak aplikacije?
4. Što predstavlja parametar primljen u `catch` bloku?
5. Kako možemo dohvatiti tekst poruke iz objekta greške?
6. Kada se izvršava `finally` blok?
7. Za koje se poslove najčešće koristi `finally`?
8. Kako se vlastita greška ručno baca pomoću naredbe `throw`?
9. Koja je razlika između `Error` i `TypeError`?
10. Kako operator `instanceof` pomaže pri različitoj obradi vrsta grešaka?

Mini zadatak

Napiši funkciju `provjeriLozinku(lozinka)`. Ako vrijednost nije string, baci `TypeError`. Ako ima manje od osam znakova, baci obični `Error`. U `catch` bloku ispiši odgovarajuću poruku, a u `finally` bloku uvijek ispiši `Provjera završena`.

JS11 - Metode za rad s nizovima

Pitanja

1. Koja je razlika između metoda `forEach()` i `map()`?
2. Što vraća `find()` ako nijedan element ne zadovoljava uvjet?
3. Što vraća `findIndex()` kada traženi element nije pronađen?
4. Kako `filter()` utječe na originalni niz?
5. Koju ulogu imaju akumulator i početna vrijednost kod metode `reduce()`?
6. Koja je razlika između metoda `some()` i `every()`?
7. Što radi `flatMap()`?
8. Po čemu se `toSorted()` razlikuje od `sort()`, a `toReversed()` od `reverse()`?
9. Kako metode `with()` i `at()` olakšavaju nepromjenjiv rad s nizom i pristup negativnim indeksima?
10. Koji dio niza vraća `slice(pocetak, kraj)` i je li završni indeks uključen?

Mini zadatak

Napravi niz objekata koji predstavljaju proizvode sa svojstvima `naziv`, `cijena` i `dostupan`. Pomoću `filter()` izdvoji dostupne proizvode, pomoću `map()` dohvati njihove nazive, a pomoću `reduce()` izračunaj ukupnu cijenu dostupnih proizvoda. Originalni niz mora ostati nepromijenjen.

JS12 - Objektno orijentirano programiranje

Pitanja

1. Koja četiri osnovna OOP principa prikazuje dokument?
2. Što je pomoćna klasa i zašto se njezine metode često deklariraju kao `static`?
3. Kako se poziva statička metoda bez stvaranja instance klase?
4. Kako se u JavaScriptu može spriječiti izravno stvaranje instance zamišljene apstraktne klase?
5. Zašto osnovna klasa može baciti grešku u metodi koju podklasa mora implementirati?
6. Čemu služe ključne riječi `extends` i `super`?
7. Što je enkapsulacija i što označava znak `#` ispred polja klase?
8. Kako getter i setter kontroliraju pristup privatnom polju?

9. Što je polimorfizam i kako ga pokazuje nadjačana metoda `opisati()`?
10. Kako metode `filter()` i `reduce()` sudjeluju u izračunu ukupnih plaća ili troškova?

Mini zadatak

Napravi osnovnu klasu `Vozilo` koju nije dopušteno izravno instancirati. Neka ima privatno polje `#godina`, getter za godinu i metodu `opis()` koju podklase moraju nadjačati. Izradi podklase `Automobil` i `Motocikl`, svaku s vlastitom implementacijom metode `opis()`.

JS13 - Generator funkcije

Pitanja

1. Kako se generator funkcija razlikuje od obične funkcije?
2. Kojom se sintaksom deklarira generator funkcija?
3. Što vraća poziv generator funkcije?
4. Što radi ključna riječ `yield`?
5. Što vraća metoda `next()` iteratora?
6. Koje značenje imaju svojstva `value` i `done` u rezultatu metode `next()`?
7. Zašto se kod generatora ne izvršava odmah do kraja?
8. Što je lazy evaluation i zašto je korisna za velike ili beskonačne nizove?
9. Kako generator može proizvesti beskonačan niz bez spremanja svih vrijednosti u memoriju?
10. Kako se vrijednost šalje natrag u generator pomoću `next(vrijednost)`?

Mini zadatak

Napiši generator `unosKorisnika()` koji redom traži ime, dob i grad. Svaki se podatak generatoru šalje pomoću `next(vrijednost)`. Nakon primitka sva tri podatka generator treba vratiti objekt `{ ime, dob, grad }`.

JS14 - Asinkroni pozivi i dohvat podataka

Pitanja

1. Što znači da je funkcija ili operacija asinkrona?
2. Što vraća funkcija `cekaj()` iz dokumenta?
3. Kako `setTimeout()` sudjeluje u razrješavanju Promisea?
4. Koja je uloga ključne riječi `await` unutar `async` funkcije?
5. Izvršavaju li se uzastopni pozivi s `await` sekvencijalno ili paralelno?
6. Što vraća funkcija `fetch()`?
7. Zašto prije poziva `odgovor.json()` treba provjeriti svojstvo `odgovor.ok`?
8. Koja je uloga metoda `.then()` i `.catch()` u lancu Promisea?

9. Kako se iz dohvaćenih podataka stvaraju `<li>` elementi i dodaju u DOM?
10. Zašto se loader prikazuje prije dohvaćanja, a skriva nakon završetka postupka?

Mini zadatak

Napravi stranicu s loaderom, praznom listom i funkcijom `dohvatiKorisnike()`. Funkcija treba pomoću `fetch()` dohvatiti korisnike s `https://jsonplaceholder.typicode.com/users`, provjeriti `response.ok`, pretvoriti odgovor u JSON i prikazati imena u `<li>` elementima. Grešku prikaži u listi, a loader sakrij u `finally` bloku.